

Лабораторная работа 4

Введение в функции. Базовая работа со строками (однобайтовыми)

1.1

Постановка задачи:

Создайте две функции, которые вычисляют факториал числа:

- функцию, которая вычисляет факториал, используя цикл;
- функцию, которая вычисляет факториал, используя рекурсивный вызов самой себя

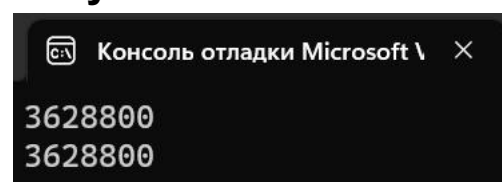
Список идентификаторов:

Имя	Тип	Смысл
fac	int	функция
Fac1	int	функция

Код программы:

```
1  #define _CRT_SECURE_NO_WARNINGS
2  #include<stdio.h>
3  #include<stdlib.h>
4  #include<math.h>
5
6  int fac(int x)
7  {
8      int c = 1;
9      for (int i = 1; i <= x; i++)
10     {
11         c = c * i;
12     }
13
14     return c;
15 }
16
17 int fac1(int x)
18 {
19     if (x == 1)
20     {
21         return 1;
22     }
23     else
24     {
25         return x * fac1(x - 1);
26     }
27 }
28
29
30 int main()
31 {
32     printf("%d\n", fac(10));
33     printf("%d\n", fac1(10));
34 }
```

Результат выполненной работы:



Консоль отладки Microsoft Visual Studio. Вывод программы:

```
3628800
3628800
```

1.2

Постановка задачи:

Объявите указатель на массив типа `int` и динамически выделите память для 12-ти элементов. Напишите функцию, которая поменяет значения чётных и нечётных ячеек массива

Список идентификаторов:

Имя	Тип	Смысл
<code>pA</code>	<code>int</code>	указатель
<code>i</code>	<code>int</code>	Аргумент цикла
<code>zum</code>	<code>void</code>	функция

Код программы:

```
1  #define _CRT_SECURE_NO_WARNINGS
2  #include<stdio.h>
3  #include<stdlib.h>
4  #include<malloc.h>
5
6  void zum(int* s)
7  {
8      int i = 0;
9      while (s[i])
10     {
11         int t = s[i];
12         s[i] = s[i + 1];
13         s[i + 1] = t;
14         i +=2;
15     }
16 }
17
18
19 int main()
20 {
21     int* pA;
22     pA = (int*)malloc(12 * sizeof(int));
23
24     for (int i = 0; i < 12; i++)
25     {
26         scanf("%d", &pA[i]);
27     }
28
29     zum(pA);
30
31     for (int i = 0; i < 12; i++)
32     {
33         printf("%d\t", pA[i]);
34     }
35
36     free(pA);
37
38
39     return 0;
40 }
41
```

Результат выполненной работы:



```
Консоль отладки Microsoft V  x  +  v
1 2 3 4 5 6 7 8 9 10 11 12
2      1      4      3      6      5      8      7      10      9      12      11
```

1.3

Постановка задачи:

Создать две основные функции:

- функцию для динамического выделения памяти под двумерный

динамический массив типа double — матрицу;

- функцию для динамического освобождения памяти под двумерный динамический массив типа double — матрицу.

Создать две вспомогательные функции:

- функцию для заполнения матрицы типа double;
- функцию для распечатки этой матрицы на экране.

Продемонстрировать работу всех этих функций в своей программе

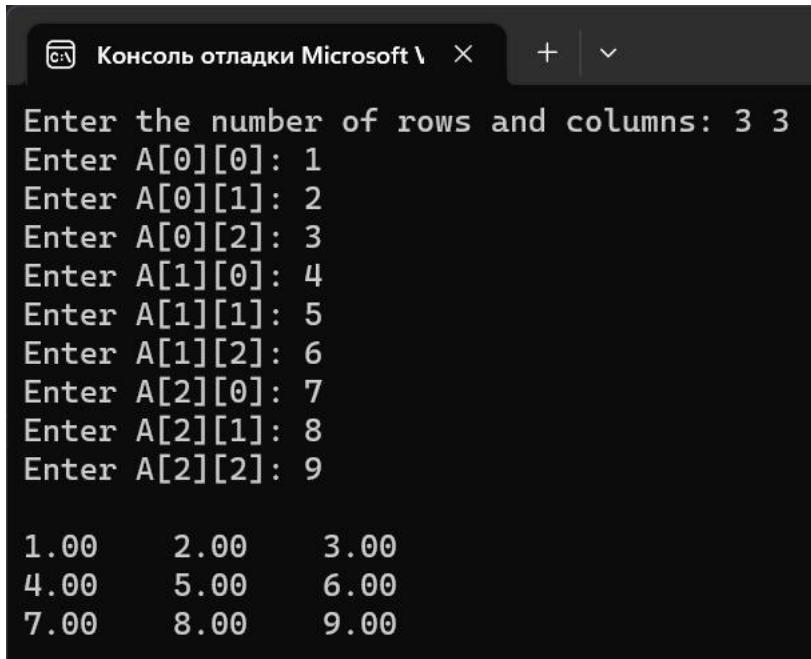
Список идентификаторов:

Имя	Тип	Смысл
allocation_matrix	double	функция
fill_matrix	void	функция
print_matrix	void	функция
free_matrix	void	функция
n	int	Входной параметр
m	int	Входной параметр
pA	double	Указатель

Код программы:

```
1  #define _CRT_SECURE_NO_WARNINGS
2  #include<stdio.h>
3  #include<stdlib.h>
4  #include<malloc.h>
5
6
7  double** allocation_matrix(int n, int m)
8  {
9      double** A = (double**)malloc(n * sizeof(double*));
10     for (int i = 0; i < n; i++)
11     {
12         A[i] = (double*)malloc(m * sizeof(double));
13     }
14
15     return A;
16 }
17
18
19 void free_matrix(double** A, int n)
20 {
21     for (int i = 0; i < n; i++)
22     {
23         free(A[i]);
24     }
25     free(A);
26 }
27
28
29 void fill_matrix(double** A, int n, int m)
30 {
31     for (int i = 0; i < n; i++)
32     {
33         for (int j = 0; j < m; j++)
34         {
35             printf("Enter A[%d][%d]: ", i, j);
36             scanf("%lf", &A[i][j]);
37         }
38     }
39     printf("\n");
40 }
41
42
43 void print_matrix(double** A, int n, int m)
44 {
45     for (int i = 0; i < n; i++)
46     {
47         for (int j = 0; j < m; j++)
48         {
49             printf("%.2lf\t", A[i][j]);
50         }
51         printf("\n");
52     }
53 }
54
55
56 int main()
57 {
58     int n, m;
59     printf("Enter the number of rows and columns: ");
60     scanf("%d%d", &n, &m);
61
62     double** pA = allocation_matrix(n, m);
63     fill_matrix(pA, n, m);
64     print_matrix(pA, n, m);
65     free_matrix(pA, n);
66
67
68     return 0;
69 }
70
71
```

Результат выполненной работы:



```
Консоль отладки Microsoft V  X + v
Enter the number of rows and columns: 3 3
Enter A[0][0]: 1
Enter A[0][1]: 2
Enter A[0][2]: 3
Enter A[1][0]: 4
Enter A[1][1]: 5
Enter A[1][2]: 6
Enter A[2][0]: 7
Enter A[2][1]: 8
Enter A[2][2]: 9

1.00    2.00    3.00
4.00    5.00    6.00
7.00    8.00    9.00
```

1.4

Постановка задачи:

Создать функцию, которая вычисляет векторное произведение двух векторов в декартовых координатах, используя указатели на соответствующие массивы.

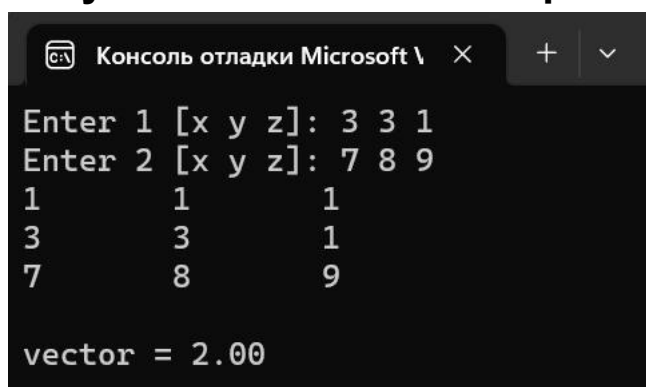
Список идентификаторов:

Имя	Тип	Смысл
allocation_matrix	int	функция
det_matrix	double	функция
print_matrix	void	функция
free_matrix	void	функция
a	int	Массив
b	int	Массив
pA	int	Указатель

Код программы:

```
1
2  #define _CRT_SECURE_NO_WARNINGS
3  #include<stdio.h>
4  #include<stdlib.h>
5  #include<malloc.h>
6
7
8  int** allocation_matrix(int* a, int* b)
9  {
10     int** A = (int**)malloc(3 * sizeof(int*));
11     for (int i = 0; i < 3; i++)
12     {
13         A[i] = (int*)malloc(3 * sizeof(int));
14     }
15     A[0][0] = 1; A[0][1] = 1; A[0][2] = 1;
16     A[1] = a;
17     A[2] = b;
18
19     return A;
20 }
21
22
23 void free_matrix(int** A)
24 {
25     for (int i = 0; i < 3; i++)
26     {
27         free(A[i]);
28     }
29     free(A);
30 }
31
32
33 void print_matrix(int** A)
34 {
35     for (int i = 0; i < 3; i++)
36     {
37         for (int j = 0; j < 3; j++)
38         {
39             printf("%d\t", A[i][j]);
40         }
41         printf("\n");
42     }
43 }
44
45 double det_matrix(int** A)
46 {
47     return ((A[0][0] * A[1][1] * A[2][2]) + (A[1][0] * A[2][1] * A[0][2]) + (A[2][0] * A[0][1] * A[1][2])) -
48            ((A[2][0] * A[1][1] * A[0][2]) + (A[0][1] * A[1][0] * A[2][2]) + (A[0][0] * A[2][1] * A[1][2]));
49 }
50
51
52
53 int main()
54 {
55     int* a = (int*)malloc(3 * sizeof(int));
56     printf("Enter 1 [x y z]: ");
57     scanf("%d%d%d", &a[0], &a[1], &a[2]);
58
59     int* b = (int*)malloc(3 * sizeof(int));
60     printf("Enter 2 [x y z]: ");
61     scanf("%d%d%d", &b[0], &b[1], &b[2]);
62
63     int** pA = allocation_matrix(a, b);
64     print_matrix(pA);
65     printf("\n");
66     printf("vector = %.2lf\n", det_matrix(pA));
67
68     free_matrix(pA);
69
70
71
72
73     return 0;
74 }
75
```


Результат выполненной работы:



```
Консоль отладки Microsoft \ x + v
Enter 1 [x y z]: 3 3 1
Enter 2 [x y z]: 7 8 9
1      1      1
3      3      1
7      8      9

vector = 2.00
```

2.1

Создайте новую программу, где с клавиатуры вводится строка некоторой длины порядка 10 латинских символов (не используйте кириллицу) в классическую строку языка C, которая имеет вид массива

`char my_string[MY_SIZE]`. `MY_SIZE` определите с помощью директивы `#define`. Значение `MY_SIZE` должно превышать длину вводимой строки с некоторым разумным запасом. Другие строки в этой задаче можете создавать либо также как статические массивы, либо как динамические массивы, но не забывайте освобождать от динамически

выделенную память с помощью функции `void free(void* ptr);`.

Выполните следующие действия и распечатайте результаты:

2.1.1

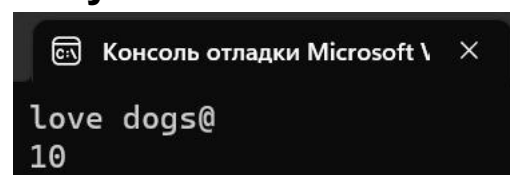
Постановка задачи:

Вычислите длину строки `my_string`, используя цикл `for`

Код программы:

```
1  #define _CRT_SECURE_NO_WARNINGS
2  #include<stdio.h>
3  #include<stdlib.h>
4  #include<string.h>
5  #include<malloc.h>
6
7
8  #define MY_SIZE 20
9
10
11 int main()
12 {
13
14     char* my_string = (char*)malloc(MY_SIZE * sizeof(char));
15     fgets(my_string, MY_SIZE, stdin);
16
17     int count = 0;
18
19     for (int i = 0; my_string[i] != '\0'; i++)
20     {
21         count += 1;
22     }
23
24     printf("%d", count - 1);
25
26     free(my_string);
27
28     return 0;
29 }
30
```

Результат выполненной работы:



```
Консоль отладки Microsoft \
love dogs@
10
```

2.1.3

Постановка задачи:

Используйте функции

size_t strlen(const char* str); или

3

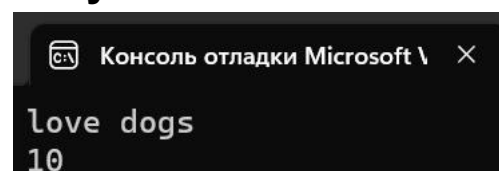
size_t strlen (const char * string, size_t maxlen); или

size_t strlen_s(const char * str, size_t strsz); для получения размера строки в виде значения

Код программы:

```
1  #define _CRT_SECURE_NO_WARNINGS
2  #include<stdio.h>
3  #include<stdlib.h>
4  #include<string.h>
5  #include<malloc.h>
6
7
8  #define MY_SIZE 20
9
10
11 int main()
12 {
13
14     char* my_string = (char*)malloc(MY_SIZE * sizeof(char));
15     fgets(my_string, MY_SIZE, stdin);
16
17
18     printf("%d", strlen(my_string));
19
20     free(my_string);
21
22     return 0;
23 }
24
```

Результат выполненной работы:



```
Консоль отладки Microsoft \
love dogs
10
```

2.1.4

Постановка задачи:

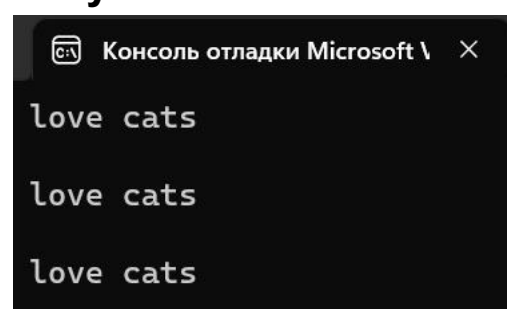
Создайте вторую строку (второй массив) и скопируйте в неё строку my_string, используя функцию

char * strcpy(char * dest, const char * src); или
char * strncpy (char * dest, const char * src, size_t n);

Код программы:

```
1  #define _CRT_SECURE_NO_WARNINGS
2  #include<stdio.h>
3  #include<stdlib.h>
4  #include<string.h>
5  #include<malloc.h>
6
7
8  #define MY_SIZE 20
9
10
11  int main()
12  {
13
14      char* my_string = (char*)malloc(MY_SIZE * sizeof(char));
15      fgets(my_string, MY_SIZE, stdin);
16
17      char* new_string = (char*)malloc(MY_SIZE * sizeof(char));
18      strcpy(new_string, my_string);
19
20      printf("\n%s\n%s\n", my_string, new_string);
21
22      free(my_string);
23      free(new_string);
24
25      return 0;
26  }
```

Результат выполненной работы:



```
Консоль отладки Microsoft \
love cats
love cats
love cats
```

2.1.5

Постановка задачи:

Создайте ещё две строки какого-либо размера и задайте их прямо в коде без клавиатуры. Сделайте конкатенацию этих двух строк, используя

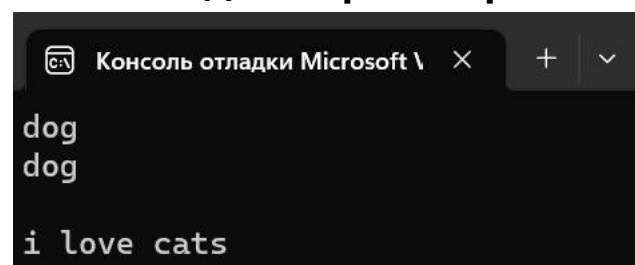
char * strcat(char * dest, const char * src); или

char * strncat(char * dest, const char * src, size_t n);. Первую строку трактуйте как dest (destination) и подберите размер этого массива с запасом

Код программы:

```
115 | #define _CRT_SECURE_NO_WARNINGS
116 | #include<stdio.h>
117 | #include<stdlib.h>
118 | #include<string.h>
119 | #include<malloc.h>
120 |
121 |
122 | #define MY_SIZE 20
123 |
124 |
125 | int main()
126 | {
127 |
128 |     char* my_string = (char*)malloc(MY_SIZE * sizeof(char));
129 |     fgets(my_string, MY_SIZE, stdin);
130 |
131 |     char* new_string = (char*)malloc(MY_SIZE * sizeof(char));
132 |     strcpy(new_string, my_string);
133 |
134 |
135 |     char newnew_string[] = "i ";
136 |     char newnewnew_string[] = "love cats";
137 |
138 |
139 |     strcat(newnew_string, newnewnew_string);
140 |
141 |     printf("%s", new_string);
142 |     printf("\n%s\n", newnew_string);
143 |
144 |     free(my_string);
145 |     free(new_string);
146 |
147 |     return 0;
148 | }
```

Список идентификаторов:



Консоль отладки Microsoft \

```
dog
dog

i love cats
```

2.2

Постановка задачи:

Конвертируйте введённые заданные как строки: число с плавающей точкой (double) и целое число (int) в значения типа double и int, используя функциями atof и atoi

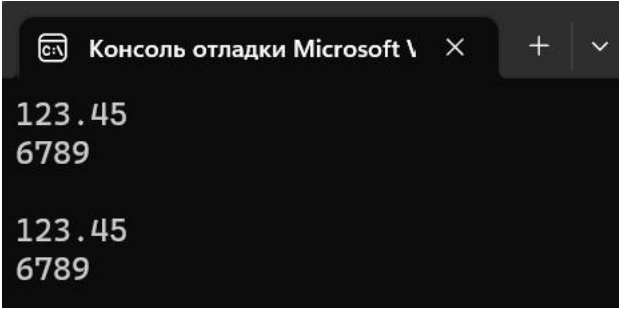
Список идентификаторов:

Имя	Тип	Смысл
MY_SIZE	const	константа
my_string	char	указатель
new_string	char	указатель

Код программы:

```
2  #define _CRT_SECURE_NO_WARNINGS
3  #include<stdio.h>
4  #include<stdlib.h>
5  #include<string.h>
6  #include<malloc.h>
7
8
9  #define MY_SIZE 20
10
11
12  int main()
13  {
14      char* my_string = (char*)malloc(MY_SIZE * sizeof(char));
15      fgets(my_string, MY_SIZE, stdin);
16
17      char* new_string = (char*)malloc(MY_SIZE * sizeof(char));
18      fgets(new_string, MY_SIZE, stdin);
19
20
21
22      printf("\n%.2lf\n", atof(my_string));
23      printf("%d\n", atoi(new_string));
24
25      free(my_string);
26      free(new_string);
27
28      return 0;
29  }
30
```

Результат выполненной работы:



```
Консоль отладки Microsoft \ X + v
123.45
6789
123.45
6789
```

2.3

Постановка задачи:

Создайте строку от 10 до 20 символов, используя только цифры, латинский буквы в разных регистрах пробельные символы и символы пунктуации

Список идентификаторов:

Имя	Тип	Смысл
MY_SIZE	const	константа
myString	Char*	указатель
l	int	Аргумент цикла

Код программы:

```
1  #define _CRT_SECURE_NO_WARNINGS
2  #include<stdio.h>
3  #include<stdlib.h>
4  #include<malloc.h>
5  #include<ctype.h>
6
7  #define MY_SIZE 20
8
9  int main()
10 {
11     char* myString = (char*)malloc(MY_SIZE * sizeof(char));
12     fgets(myString, MY_SIZE, stdin);
13
14     tolower(myString);
15
16     for (int i = 0; i < strlen(myString); i++)
17     {
18         if (isalnum(myString[i]) == 1)
19             printf("%c - num\n", myString[i]);
20         if (isalpha(myString[i]))
21             printf("%c - letter\n", myString[i]);
22         if (isspace(myString[i]))
23             printf("%c - spase\n", myString[i]);
24         if (isblank(myString[i]))
25             printf("%c - 0_0\n", myString[i]);
26     }
27
28
29     return 0;
30 }
```


Результат выполненной работы:

```
Консоль отладки Microsoft \ × + ▾  
i love cats 333  
i - letter  
  - spase  
  - 0_0  
l - letter  
o - letter  
v - letter  
e - letter  
  - spase  
  - 0_0  
c - letter  
a - letter  
t - letter  
s - letter  
  - spase  
  - 0_0  
  
- spase
```